Jazyk Python

Zložitosť algoritmov, optimalizácia, dynamické programovanie
prednáška 6

Ústav umelej inteligencie
Technická univerzita v Košiciach
Ing. Ján Magyar, PhD.

Algoritmizácia

- popis riešenia problému ako postupnosť jednoznačných krokov
- každý problém má niekoľko riešení – ako vybrať to najlepšie?
- výber vhodného algoritmu môže byť rozdiel medzi vyriešeným/neriešiteľným problémom
- väčšinu problémov vieme namapovať do problému, ktorý je možné vyriešiť niektorým z klasických algoritmov

Analýza algoritmov - počet operácií

```
def exp1(a, b):
    ans = 1
    while (b > 0):
        ans *= a
        b -= 1
    return ans
```

Analýza algoritmov - asymptotická notácia

- vyjadruje horný limit zložitosti algoritmu ak vstup je stále väčší a väčší
- najčastejšie sa používa Big-O notácia
- zanedbávajú sa konštantné časti algoritmu, do úvahy sa berie iba časť, ktorá je závislá od veľkosti vstupných parametrov
- $f(x) \in O(n^2)$

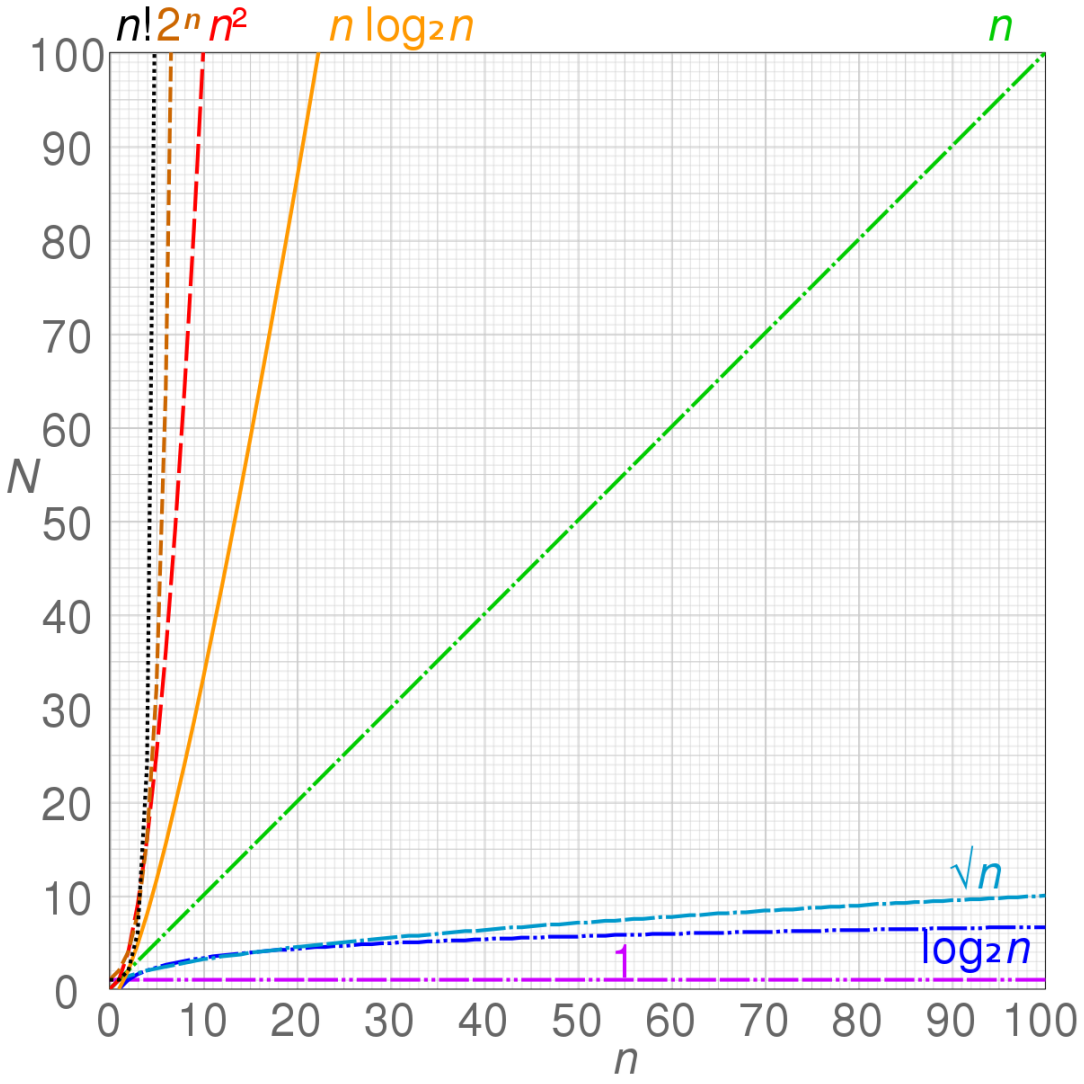
Výpočet mocnín

```
def exp2(a, b):
    if b == 1:
        return a
    if (b % 2) == 0:
        return exp2(a * a, b / 2)
    else:
        return a * exp2(a, b - 1)
```

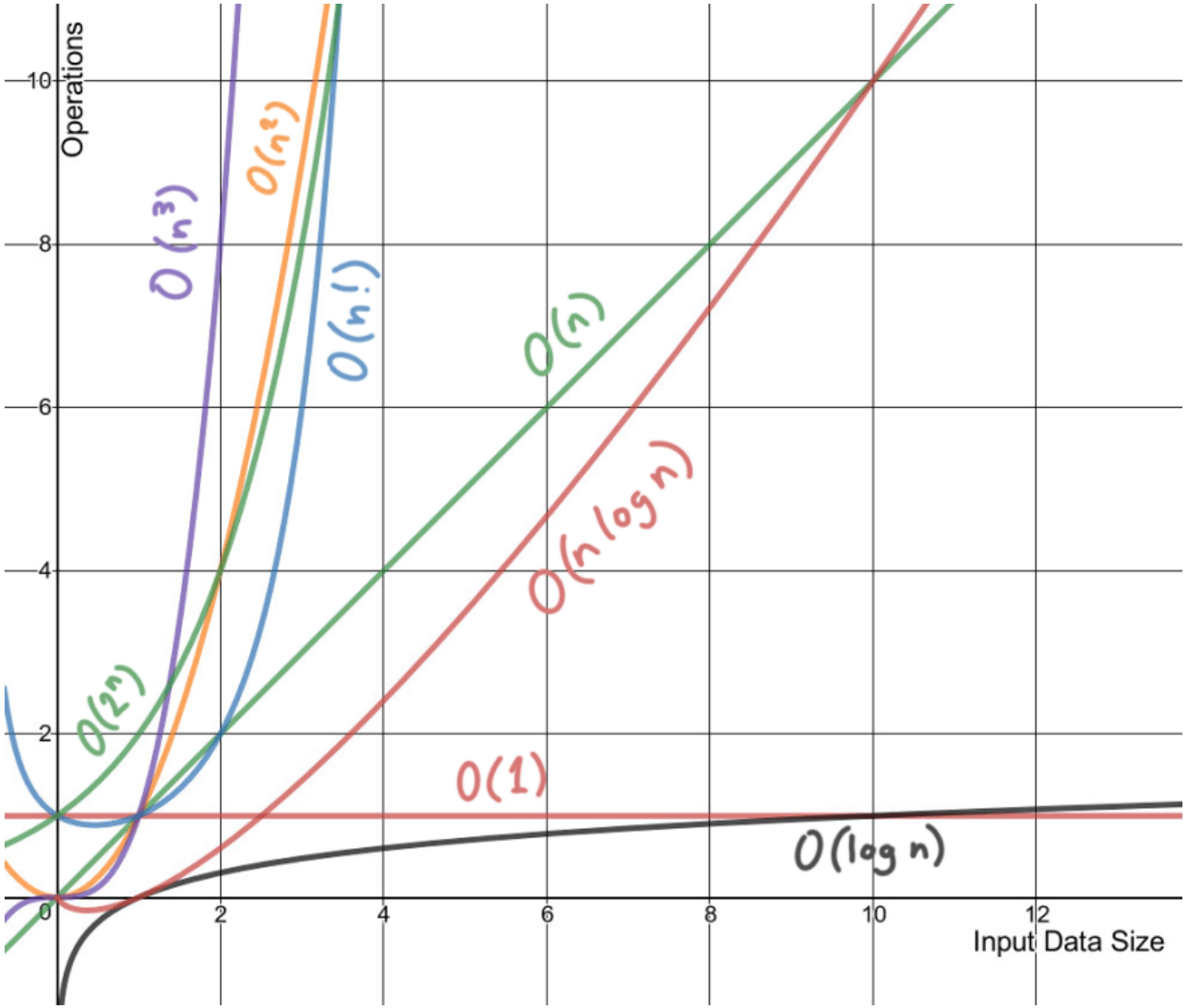
Fibonacciho čísla

```
def fib(a):
    if a == 0 or a == 1:
        return 1
    else:
        return fib(a - 1) + fib(a - 2)
```

Časová komplexita algoritmov



Časová komplexita algoritmov



Časová komplexita algoritmov

predpokladáme frekvenciu 1GHz (jedna operácia za nanosekundu)

	n = 1000	n = 1,000,000,000
$O(\log n)$	10 ns	10 ms
$O(n)$	1 μs	1 s
$O(n^2)$	1 ms	16 minút
$O(2^n)$	10^{284} rokov	...

Optimalizácia kódu

- cieľom je zvýšiť výkon existujúceho programu
- súčasť ladenia kódu
- eliminácia zbytočných a viacnásobných operácií

Viacnásobné prechádzanie zoznamom

```
my_list = [1240, -25, 37.24, -12, 0, 35000, 24, 17.23]

for x in range(len(my_list)):
    if my_list[x] < 0:
        my_list[x] = 0

for x in range(len(my_list)):
    my_list[x] *= 1.05

print(my_list)
```

Zbytočné operácie

```
def is_prime(number):
    for x in range(2, number):
        if number % x == 0:
            return False
    return True


is_prime(123475862311)
```

Fibonacciho čísla – ešte raz

```
def fib(a):
    if a == 0 or a == 1:
        return 1
    else:
        return fib(a - 1) + fib(a - 2)
```

Dynamické programovanie

- slúži pre optimalizáciu problémov s exponenciálnou zložitosťou
- aplikácia
  - pretínajúce sa podproblémy – Fibonacciho čísla
  - optimálna štruktúra – knapsack problem

Fibonacciho čísla

```
steps = 0

def fib(a):
    global steps
    steps += 1
    print("Calculating fib for", a)
    if a == 0 or a == 1:
        return 1
    else:
        return fib(a - 1) + fib(a - 2)
```

Fibonacciho čísla - jednoduchšie

```
def fib_smart(a, memo):
    global num_calls

    num_calls += 1
    print("fib_smart called with", a)
    if a not in memo:
        memo[a] = fib_smart(a - 2, memo) + fib_smart(a - 1, memo)
    return memo[a]

n = 10
memo = {0: 1, 1: 1}
num_calls = 0
fib_smart(n, memo)
```

Memoizácia

- čiastočné výsledky ukladáme do tabuľky
- ak sme ešte nevypočítali výsledok pre daný vstup, vypočítame a pridáme ho do tabuľky
- pri opätovnom volaní funkcie iba načítame výsledok z tabuľky (table lookup)
- všetky volania funkcie musia pracovať s tou istou tabuľkou

Teoréma optimálnej štruktúry

Ak riešenia podproblémov sú lokálne optimálne, potom celkové riešenie bude globálne optimálne.

Knapsack problem (problém batohu)

- klasický problém optimalizačných algoritmov
- zlodej sa vlámal do bytu a chce ukradnúť cennosti v najvyššej možnej hodnote
- zlodej má iba jeden batoh s maximálnou nosnosťou `W`
- každý objekt v byte je popísaný dvojicou `(m, w)` kde `m` je hodnota objektu a `w` je jeho hmotnosť
- úlohou je nájsť množinu najcennejších objektov, ktorých celková hmotnosť nepresiahne `W`

Greedy riešenie knapsack problému

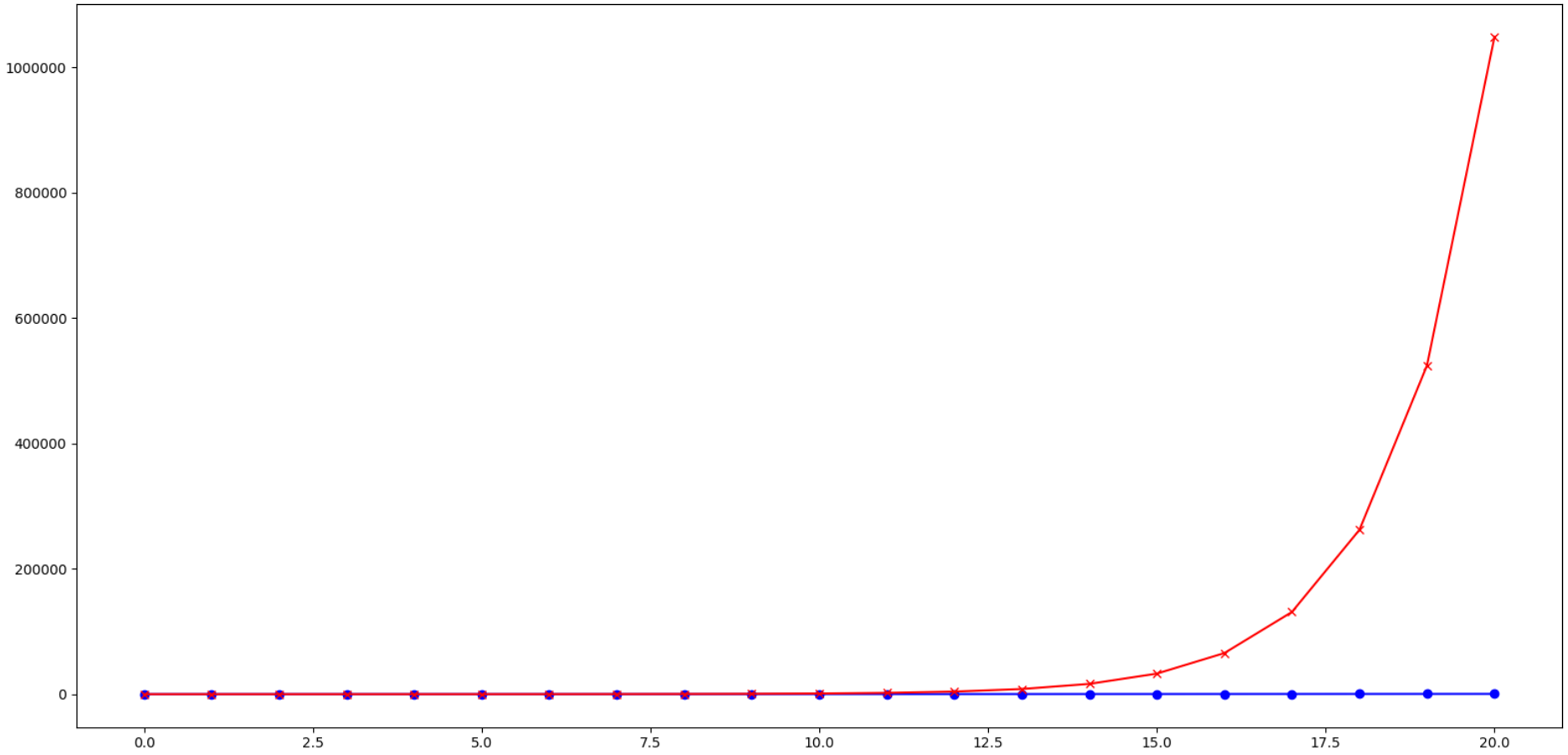
- greedy algoritmy hľadajú vždy najlepšie okamžité riešenie
- riešenie: zober najcennejšie veci až dovtedy, kým tvoj batoh nie je plný
- nie vždy nájdeme optimálne riešenie, ale riešenie je zvyčajne dosť dobré
- jednoduchá implementácia, výpočtovo lacné

príklad:
w = [5, 3, 2]
m = [9, 7, 8]
W = 5

Riešenie knapsack problému

- intuitívne – brute force
- vyskúšame všetky možnosti
- možné riešenia reprezentujeme ako vektor n hodnôt
- každá hodnota bude 0 alebo 1 (nezoberieme/zoberieme prvok)
- pre každý vektor skontrolujeme hmotnosť prvkov
- koľko máme možností?

Prečo nepoužívame algoritmy s exponenciálnym rastom?



Riešenie knapsack problému

- vhodná reprezentácia – rozhodovací strom
  - každý uzol je reprezentovaný ako trojica `(i, w, m)`
  - ľavá vetva obsahuje prípady, kde sme nezobrali objekt s indexom `i`
  - pravá vetva obsahuje prípady, kde sme zobrali objekt s indexom `i`
  - optimálne riešenie je listový uzol s najväčšou hodnotou `m`

príklad:
w = [5, 3, 2]
m = [9, 7, 8]
W = 5

```
def max_val(w, m, i, aW):
    global num_calls
    num_calls += 1
    if i == 0:
        if w[i] <= aW:
            return m[i]
        else:
            return 0
    without_i = max_val(w, m, i - 1, aW)
    if w[i] > aW:
        return without_i
    else:
        with_i = m[i] + max_val(w, m, i - 1, aW - w[i])
    return max(with_i, without_i)
```

```
def fast_max_val(w, v, i, aW, m):
    global num_calls
    num_calls += 1
    try:
        return m[(i, aW)]
    except KeyError:
        if i == 0:
            if w[i] <= aW:
                m[(i, aW)] = v[i]
                return v[i]
            else:
                m[(i, aW)] = 0
                return 0
        without_i = fast_max_val(w, v, i - 1, aW, m)
        if w[i] > aW:
            m[(i, aW)] = without_i
            return without_i
        else:
            with_i = v[i] + fast_max_val(w, v, i - 1, aW - w[i], m)
        res = max(with_i, without_i)
        m[(i, aW)] = res
        return res
```

Zhrnutie

- analýza zložitosti algoritmov
- big-O notácia, skupiny zložitosti algoritmov
- optimalizácia kódu a jej ciele
- dynamické programovanie
- memoizácia
- teoréma optimálnej štruktúry
- rozhodovací strom v riešení algoritmizačných úloh